

1 Configurações do experimento

1.1 Hardware

Tabela 1: Configuração de hardware da máquina utilizada.

Característica	Configuração
Processador	Intel Core i5-10300H @ 2.50 GHz
Arquitetura	x86-64
Núcleos físicos	4
Threads de hardware	8
Cache L2 total reportada	1024 KiB (1 MiB)
Cache L3	8192 KiB (8 MiB)

1.2 Software

Tabela 2: Configuração do ambiente de software.

Característica	Configuração
Sistema	Windows
Ambiente de compilação	MSYS2 UCRT64
Compilador	GCC 15.1.0
Arquitetura de compilação	x86-64
Linguagem	C
Padrão da linguagem	C11
Relógio utilizado	CLOCK_MONOTONIC
Tipo de dado	int
Tamanho de <code>int</code>	4 bytes

1.3 Parâmetros do benchmark

Tabela 3: Parâmetros utilizados nas execuções.

Parâmetro	Valor
Execuções de aquecimento	3
Medições válidas	20
Acumuladores da versão Multi4	4
Estatística utilizada	Mediana
Unidade de tempo	Milissegundos
Relógio	CLOCK_MONOTONIC
Métrica de desempenho	Speedup = Single / Multi4

1.4 Tamanhos dos vetores

Considerando `sizeof(int) = 4 bytes`, os tamanhos utilizados no benchmark são apresentados na Tabela 4.

Tabela 4: Dimensões dos vetores utilizados no experimento.

N	Tamanho
32.768	128 KiB
65.536	256 KiB
262.144	1 MiB
1.048.576	4 MiB
2.097.152	8 MiB
4.194.304	16 MiB
16.777.216	64 MiB

1.5 Configuração de compilação

O código utiliza a macro `OPT_LEVEL` para registrar no próprio output o nível de otimização utilizado na compilação.

Compilação sem otimização:

```
gcc -O0 -std=c11 -Wall -Wextra -pedantic \
-DOPT_LEVEL=\"00\" ATVD2.c -o ATVD2_00.exe
```

Compilação com -O2:

```
gcc -O2 -std=c11 -Wall -Wextra -pedantic \
-DOPT_LEVEL=\"02\" ATVD2.c -o ATVD2_02.exe
```

Compilação com -O3:

```
gcc -O3 -std=c11 -Wall -Wextra -pedantic \
-DOPT_LEVEL=\"03\" ATVD2.c -o ATVD2_03.exe
```

2 Código-fonte

```
1 #define _POSIX_C_SOURCE 200809L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <stdint.h>
6 #include <stddef.h>
7 #include <time.h>
8 #include <string.h>
9
10 #ifndef OPT_LEVEL
11 #define OPT_LEVEL "unknown"
12 #endif
13
14 #define WARMUP_RUNS 3
15 #define MEASURE_RUNS 20
16 #define NUM_ACCUMULATORS 4
17
18 #if defined(__GNUC__)
19 #define NOINLINE __attribute__((noinline))
20 #define COMPILER_BARRIER() \
21     __asm__ __volatile__("" ::: "memory")
22 #else
23 #define NOINLINE
24 #define COMPILER_BARRIER() ((void)0)
25 #endif
26
27
28 /*
29  * Mantém os resultados das operações observáveis durante a
30  * execução e impede a eliminação dos cálculos pelo compilador.
31  */
32 static volatile int benchmark_sink = 0;
33
34
35 /*
36  * Define os tamanhos de vetor utilizados no experimento.
37  *
38  * Considerando sizeof(int) = 4 bytes:
39  *
40  *      32.768 elementos      = 128 KiB
41  *      65.536 elementos      = 256 KiB
42  *      262.144 elementos     = 1 MiB
43  *      1.048.576 elementos    = 4 MiB
44  *      2.097.152 elementos    = 8 MiB
45  *      4.194.304 elementos    = 16 MiB
46  *      16.777.216 elementos   = 64 MiB
47  */
48 static const size_t N_VALUES[] = {
49     32768,
50     65536,
51     262144,
52     1048576,
53     2097152,
54     4194304,
55     16777216
56 };
57
58 #define NUM_N_VALUES \
59     (sizeof(N_VALUES) / sizeof(N_VALUES[0]))
60
61
62 /* =====
63  * TEMPORIZAÇÃO
64  * ===== */
65
66 /*
67  * Calcula o intervalo entre dois instantes obtidos por
68  * clock_gettime() e retorna o tempo em milissegundos.
69  */
70 static double elapsed_ms(
```

```

71     const struct timespec *start,
72     const struct timespec *end
73 )
74 {
75     double seconds =
76         (double)(end->tv_sec - start->tv_sec);
77
78     double nanoseconds =
79         (double)(end->tv_nsec - start->tv_nsec);
80
81     return (seconds * 1000.0)
82         + (nanoseconds / 1000000.0);
83 }
84
85
86 /* =====
87  * INICIALIZAÇÃO DO VETOR
88  * ===== */
89
90 /*
91  * Inicializa os elementos do vetor com valores inteiros
92  * periódicos no intervalo de 1 a 8.
93  *
94  * Cada posição é escrita independentemente das demais.
95  */
96 static NOINLINE void initialize_vector(
97     int *vector,
98     size_t n
99 )
100 {
101     for (size_t i = 0; i < n; ++i) {
102         vector[i] = (int)((i & 7u) + 1u);
103     }
104 }
105
106
107 /* =====
108  * SOMA COM UM ACUMULADOR
109  * ===== */
110
111 /*
112  * Realiza a redução do vetor utilizando um único acumulador.
113  *
114  * A atualização de sum em cada iteração depende do resultado
115  * produzido na iteração anterior, formando uma única cadeia
116  * de dependência.
117  */
118 static NOINLINE int sum_single_accumulator(
119     const int *vector,
120     size_t n
121 )
122 {
123     int sum = 0;
124
125     for (size_t i = 0; i < n; ++i) {
126         sum += vector[i];
127     }
128
129     return sum;
130 }
131
132
133 /* =====
134  * SOMA COM QUATRO ACUMULADORES
135  * ===== */
136
137 /*
138  * Realiza a redução utilizando quatro acumuladores
139  * independentes.
140  *
141  * Os quatro acumuladores estabelecem cadeias de dependência
142  * distintas e aumentam o paralelismo em nível de instrução
143  * disponível durante a execução do laço.

```

```

144  */
145  static NOINLINE int sum_four_accumulators(
146      const int *vector,
147      size_t n
148  )
149  {
150      int sum0 = 0;
151      int sum1 = 0;
152      int sum2 = 0;
153      int sum3 = 0;
154
155      size_t i = 0;
156
157      for (; i + 3 < n; i += 4) {
158          sum0 += vector[i];
159          sum1 += vector[i + 1];
160          sum2 += vector[i + 2];
161          sum3 += vector[i + 3];
162      }
163
164      /*
165       * Processa elementos residuais quando n não é múltiplo
166       * de quatro.
167       */
168      for (; i < n; ++i) {
169          sum0 += vector[i];
170      }
171
172      return sum0 + sum1 + sum2 + sum3;
173  }
174
175
176  /* =====
177   * MEDIÇÃO DA INICIALIZAÇÃO
178   * ===== */
179
180  /*
181   * Mede exclusivamente o intervalo correspondente à
182   * inicialização do vetor.
183   */
184  static double measure_initialization(
185      int *vector,
186      size_t n
187  )
188  {
189      struct timespec start;
190      struct timespec end;
191
192      COMPILER_BARRIER();
193
194      if (clock_gettime(
195          CLOCK_MONOTONIC,
196          &start
197      ) != 0) {
198
199          perror("clock_gettime");
200          exit(EXIT_FAILURE);
201      }
202
203      COMPILER_BARRIER();
204
205      initialize_vector(vector, n);
206
207      COMPILER_BARRIER();
208
209      if (clock_gettime(
210          CLOCK_MONOTONIC,
211          &end
212      ) != 0) {
213
214          perror("clock_gettime");
215          exit(EXIT_FAILURE);
216      }

```

```

217     COMPILER_BARRIER();
218
219
220     return elapsed_ms(&start, &end);
221 }
222
223
224 /* =====
225  * MEDIÇÃO DA SOMA COM UM ACUMULADOR
226  * ===== */
227
228 /*
229  * Mede exclusivamente o tempo da redução realizada com um
230  * único acumulador.
231  */
232 static double measure_single(
233     const int *vector,
234     size_t n,
235     int *result
236 )
237 {
238     struct timespec start;
239     struct timespec end;
240
241     COMPILER_BARRIER();
242
243     if (clock_gettime(
244         CLOCK_MONOTONIC,
245         &start
246     ) != 0) {
247
248         perror("clock_gettime");
249         exit(EXIT_FAILURE);
250     }
251
252     COMPILER_BARRIER();
253
254     *result =
255         sum_single_accumulator(
256             vector,
257             n
258         );
259
260     COMPILER_BARRIER();
261
262     if (clock_gettime(
263         CLOCK_MONOTONIC,
264         &end
265     ) != 0) {
266
267         perror("clock_gettime");
268         exit(EXIT_FAILURE);
269     }
270
271     COMPILER_BARRIER();
272
273     return elapsed_ms(&start, &end);
274 }
275
276
277 /* =====
278  * MEDIÇÃO DA SOMA COM QUATRO ACUMULADORES
279  * ===== */
280
281 /*
282  * Mede exclusivamente o tempo da redução realizada com quatro
283  * acumuladores independentes.
284  */
285 static double measure_multi4(
286     const int *vector,
287     size_t n,
288     int *result
289 )

```

```

290 {
291     struct timespec start;
292     struct timespec end;
293
294     COMPILER_BARRIER();
295
296     if (clock_gettime(
297         CLOCK_MONOTONIC,
298         &start
299     ) != 0) {
300
301         perror("clock_gettime");
302         exit(EXIT_FAILURE);
303     }
304
305     COMPILER_BARRIER();
306
307     *result =
308         sum_four_accumulators(
309             vector,
310             n
311         );
312
313     COMPILER_BARRIER();
314
315     if (clock_gettime(
316         CLOCK_MONOTONIC,
317         &end
318     ) != 0) {
319
320         perror("clock_gettime");
321         exit(EXIT_FAILURE);
322     }
323
324     COMPILER_BARRIER();
325
326     return elapsed_ms(&start, &end);
327 }
328
329
330 /* =====
331  * ESTATÍSTICA
332  * ===== */
333
334 /*
335  * Implementa a comparação utilizada por qsort() para ordenar
336  * os tempos armazenados como valores do tipo double.
337  */
338 static int compare_double(
339     const void *a,
340     const void *b
341 )
342 {
343     double x = *(const double *)a;
344     double y = *(const double *)b;
345
346     if (x < y) {
347         return -1;
348     }
349
350     if (x > y) {
351         return 1;
352     }
353
354     return 0;
355 }
356
357
358 /*
359  * Calcula a mediana dos valores fornecidos.
360  */
361 static double median(
362     double values[],

```

```

363     size_t count
364 )
365 {
366     qsort(
367         values,
368         count,
369         sizeof(double),
370         compare_double
371     );
372
373     if ((count % 2) == 0) {
374
375         return (
376             values[(count / 2) - 1]
377             +
378             values[count / 2]
379         ) / 2.0;
380     }
381
382     return values[count / 2];
383 }
384
385
386 /* =====
387  * VALIDAÇÃO DO RESULTADO
388  * ===== */
389
390 /*
391  * Calcula o resultado esperado para o padrão utilizado na
392  * inicialização.
393  *
394  * A sequência periódica
395  *
396  *     1 2 3 4 5 6 7 8
397  *
398  * possui soma igual a 36 para cada bloco de oito elementos.
399  */
400 static int expected_sum(size_t n)
401 {
402     long long value =
403         (long long)(n / 8u) * 36LL;
404
405     if (value > INT32_MAX) {
406
407         fprintf(
408             stderr,
409             "Erro: soma esperada excede "
410             "o limite de int.\n"
411         );
412
413         exit(EXIT_FAILURE);
414     }
415
416     return (int)value;
417 }
418
419
420 /* =====
421  * BENCHMARK PARA UM VALOR DE N
422  * ===== */
423
424 /*
425  * Executa o procedimento experimental para um tamanho
426  * específico de vetor.
427  */
428 static int run_for_n(size_t n)
429 {
430     size_t bytes =
431         n * sizeof(int);
432
433     int *vector =
434         malloc(bytes);
435

```

```

436     if (vector == NULL) {
437
438         fprintf(
439             stderr,
440             "Erro: nao foi possivel "
441             "alocar %zu bytes.\n",
442             bytes
443         );
444
445         return 0;
446     }
447
448
449     /*
450     * Materializa previamente as páginas de memória associadas
451     * ao bloco alocado antes do início das medições.
452     */
453     memset(
454         vector,
455         0,
456         bytes
457     );
458
459     COMPILER_BARRIER();
460
461
462     /*
463     * Executa as etapas de aquecimento.
464     *
465     * Os tempos dessas execuções não são incluídos no resultado
466     * final do benchmark.
467     */
468     for (
469         int w = 0;
470         w < WARMUP_RUNS;
471         ++w
472     ) {
473
474         initialize_vector(
475             vector,
476             n
477         );
478
479         int warm_single =
480             sum_single_accumulator(
481                 vector,
482                 n
483             );
484
485         int warm_multi =
486             sum_four_accumulators(
487                 vector,
488                 n
489             );
490
491         if (warm_single != warm_multi) {
492
493             fprintf(
494                 stderr,
495                 "Erro durante warm-up "
496                 "para N=%zu.\n",
497                 n
498             );
499
500             free(vector);
501
502             return 0;
503         }
504
505         benchmark_sink =
506             warm_single;
507
508         benchmark_sink =

```

```

509         warm_multi;
510     }
511
512     double initialization_times[MEASURE_RUNS];
513     double single_times[MEASURE_RUNS];
514     double multi_times[MEASURE_RUNS];
515
516     int last_single = 0;
517     int last_multi = 0;
518
519
520
521     /*
522      * Executa as medições válidas.
523      *
524      * A ordem entre Single e Multi4 é alternada a cada
525      * repetição para reduzir possíveis vantagens sistemáticas
526      * relacionadas ao estado da hierarquia de memória.
527      */
528     for (
529         int r = 0;
530         r < MEASURE_RUNS;
531         ++r
532     ) {
533
534         initialization_times[r] =
535             measure_initialization(
536                 vector,
537                 n
538             );
539
540         int single_result = 0;
541         int multi_result = 0;
542
543
544         if ((r % 2) == 0) {
545
546             single_times[r] =
547                 measure_single(
548                     vector,
549                     n,
550                     &single_result
551                 );
552
553             benchmark_sink =
554                 single_result;
555
556
557             multi_times[r] =
558                 measure_multi4(
559                     vector,
560                     n,
561                     &multi_result
562                 );
563
564             benchmark_sink =
565                 multi_result;
566
567         } else {
568
569             multi_times[r] =
570                 measure_multi4(
571                     vector,
572                     n,
573                     &multi_result
574                 );
575
576             benchmark_sink =
577                 multi_result;
578
579
580             single_times[r] =
581                 measure_single(

```

```

582         vector,
583         n,
584         &single_result
585     );
586
587     benchmark_sink =
588         single_result;
589 }
590
591
592 /*
593  * Valida a equivalência entre os resultados produzidos
594  * pelas duas implementações.
595  */
596 if (single_result != multi_result) {
597
598     fprintf(
599         stderr,
600         "Erro: resultados diferentes "
601         "para N=%zu "
602         "(single=%d, multi=%d).\n",
603         n,
604         single_result,
605         multi_result
606     );
607
608     free(vector);
609
610     return 0;
611 }
612
613 last_single =
614     single_result;
615
616 last_multi =
617     multi_result;
618 }
619
620
621 /*
622  * Valida o resultado final em relação ao valor esperado.
623  */
624 int expected =
625     expected_sum(n);
626
627 if (
628     last_single != expected
629     ||
630     last_multi != expected
631 ) {
632
633     fprintf(
634         stderr,
635         "Erro de validacao para N=%zu. "
636         "Esperado=%d, single=%d, multi=%d\n",
637         n,
638         expected,
639         last_single,
640         last_multi
641     );
642
643     free(vector);
644
645     return 0;
646 }
647
648
649 /*
650  * Calcula as medianas das medições realizadas.
651  */
652 double init_median =
653     median(
654         initialization_times,

```

```

655     MEASURE_RUNS
656 );
657
658 double single_median =
659     median(
660         single_times,
661         MEASURE_RUNS
662     );
663
664 double multi_median =
665     median(
666         multi_times,
667         MEASURE_RUNS
668     );
669
670
671 /*
672  * Calcula o speedup da implementação Multi4 em relação
673  * à implementação Single.
674  */
675 double speedup = 0.0;
676
677 if (multi_median > 0.0) {
678
679     speedup =
680         single_median
681         /
682         multi_median;
683 }
684
685
686 /*
687  * Converte o tamanho do vetor para MiB.
688  */
689 double size_mib =
690     (double)bytes
691     /
692     (1024.0 * 1024.0);
693
694
695 printf(
696     "%-11zu %10.3f %12.6f "
697     "%14.6f %14.6f %9.3fx\n",
698     n,
699     size_mib,
700     init_median,
701     single_median,
702     multi_median,
703     speedup
704 );
705
706
707 free(vector);
708
709 return 1;
710 }
711
712
713 /* =====
714  * PROGRAMA PRINCIPAL
715  * ===== */
716
717 int main(void)
718 {
719     /*
720      * Apresenta as configurações gerais do benchmark.
721      */
722     printf("\n");
723     printf("ILP Benchmark\n");
724
725     printf(
726         "Optimization : %s\n",
727         OPT_LEVEL

```

```

728 );
729
730 printf(
731     "Data type      : int (%zu bytes)\n",
732     sizeof(int)
733 );
734
735 printf(
736     "Clock          : CLOCK_MONOTONIC\n"
737 );
738
739 printf(
740     "Warm-ups       : %d\n",
741     WARMUP_RUNS
742 );
743
744 printf(
745     "Measurements   : %d\n",
746     MEASURE_RUNS
747 );
748
749 printf(
750     "Accumulators   : %d\n",
751     NUM_ACCUMULATORS
752 );
753
754 printf("\n");
755
756
757 /*
758  * Apresenta o cabeçalho da tabela de resultados.
759  */
760 printf(
761     "%-11s %10s %12s "
762     "%14s %14s %9s\n",
763     "N",
764     "Size(MiB)",
765     "Init(ms)",
766     "Single(ms)",
767     "Multi4(ms)",
768     "Speedup"
769 );
770
771 printf(
772     "-----"
773     "-----"
774     "-----\n"
775 );
776
777
778 /*
779  * Executa o benchmark para todos os tamanhos definidos.
780  */
781 for (
782     size_t i = 0;
783     i < NUM_N_VALUES;
784     ++i
785 ) {
786
787     if (!run_for_n(N_VALUES[i])) {
788
789         printf(
790             "\nStatus: ERROR\n"
791         );
792
793         return EXIT_FAILURE;
794     }
795 }
796
797
798 printf(
799     "-----"
800     "-----"

```

```
801     "-----\n"
802 );
803
804 printf("Status: OK\n");
805
806 printf(
807     "Sink: %d\n",
808     benchmark_sink
809 );
810
811 printf("\n");
812
813 return EXIT_SUCCESS;
814 }
```

Listing 1: Código-fonte do benchmark de ILP.